SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

*I **B Ragava Prasath (Reg No: 192421026)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

SOLUTIONS

Solution 1 — Equivalence Partitioning & Boundary Value Errors (Online Shopping Input)

Errors Identified

- Valid and invalid inputs were mixed into a single equivalence class, violating the rule that each class must contain only inputs expected to be processed the same way by the system.
- Boundary values (the edges of each valid/invalid range) were never tested, so off-by-one defects at the limits would not be caught.
- No distinction was made between valid partitions and invalid partitions, so invalid-input handling (error messages, rejection logic) is untested.

Root Cause Analysis

The root cause is an incomplete understanding of class definition: equivalence partitioning requires that classes be built from the specification's valid and invalid ranges separately, not from convenience groupings of 'similar-looking' values. Because the tester treated the input domain as one continuous range, the partitioning lost its purpose of reducing redundant tests while preserving defect-detection power. The absence of BVA compounds this: BVA was likely skipped because the tester assumed interior values were 'representative enough', ignoring that boundaries are where off-by-one and comparison-operator defects (<, <=, >, >=) concentrate.

Corrected Test Design

Assume a quantity field accepting 1–10 units per order. The corrected partitions are:

- Valid class: 1–10 (test a mid value, e.g., 5)
- Invalid class (below range): values ≤ 0 (test 0 and -1)
- Invalid class (above range): values ≥ 11 (test 11 and a large value)
- Boundary values: 1, 10 (valid boundaries), 0, 11 (invalid boundaries just outside range)

Justification

Separating valid and invalid classes ensures each test has a single, predictable expected outcome, making pass/fail verdicts unambiguous. Adding boundary-value test cases targets the exact locations where comparison-operator defects occur, which interior-only testing would never expose. Together, this raises defect-detection effectiveness while keeping the test suite minimal, since each class and boundary is covered by one representative case rather than exhaustively testing every possible value.

Solution 2 — Unstructured Random Testing on a Banking Application

Errors Identified

- Testing relied on random, ad-hoc input selection instead of a systematic black-box technique, so coverage of the input domain was accidental rather than guaranteed.
- "High coverage" was claimed without a defined coverage criterion (no mapping to equivalence classes, boundaries, or business rules), making the claim unverifiable.
- Transaction-limit boundaries (minimum/maximum transfer amount, daily limit) were never deliberately targeted, so defects at these critical points were missed.

Root Cause Analysis

The root cause is a strategic failure to select a testing technique appropriate to a high-risk, rule-driven domain. Banking transaction limits are precisely the kind of discrete, boundary-governed business rules that random testing

is statistically unlikely to hit, because random inputs are far more likely to fall in the middle of a range than exactly on a limit. The team also lacked a coverage model, so there was no objective way to know which rules had and had not been exercised — coverage was assumed rather than measured.

Recommended Black-Box Techniques

- Equivalence Class Partitioning: classes for valid transfer amounts, below-minimum, above-maximum, and invalid formats.
- Boundary Value Analysis: test exactly at and just outside the minimum and maximum transaction/daily limits.
- Decision Table Testing: for combined rules such as account type + transaction type + daily limit already used, to capture combinations the team is currently missing.
- State Transition Testing: for account states (active, frozen, overdrawn) that affect whether a transaction is permitted.

Justification

These techniques replace guesswork with a traceable mapping from specification to test case, so every business rule and limit has at least one dedicated test. This guarantees that critical boundaries are exercised deliberately rather than by chance, giving a defensible, repeatable basis for the coverage claim and substantially improving the likelihood of catching limit-related defects before release.

Solution 3 — Incomplete Control Flow Graph in White-Box Testing

Errors Identified

- Decision nodes (if/else, switch branches) were omitted from the CFG, so the graph does not represent all possible execution branches.
- Loop constructs were not modeled with their back-edges, meaning repeated execution paths are missing from the graph.
- Because the graph is incomplete, the independent paths derived using McCabe's cyclomatic complexity (V(G) = E – N + 2) will be undercounted, so some execution paths are never tested.

Root Cause Analysis

The underlying cause is treating control flow graph construction as a sequential diagram of statements rather than a true graph of all decision-driven branches. Every point in the code where control can diverge (conditionals) or repeat (loops) must become a node with outgoing edges for each possible outcome. Skipping decision nodes typically happens when nested or compound conditions are collapsed into a single node, and skipping loop back-edges happens when the loop body is drawn only once instead of as a cycle returning to the loop-condition node.

Corrected Control Flow Representation

For a module with one if-else inside a while loop, the corrected CFG has: a loop-condition node with two outgoing edges (continue into the body / exit the loop), a decision node inside the body with two outgoing edges (if-true / else-false), and a back-edge from the end of the loop body returning to the loop-condition node. Applying cyclomatic complexity V(G) = E – N + 2 to this corrected graph (e.g., 7 edges, 6 nodes → V(G) = 3) identifies 3 independent paths: (1) loop never executes, (2) loop executes once via the if-branch, (3) loop executes once via the else-branch — with additional paths for multiple iterations covered by re-tracing the back-edge.

Justification

A complete CFG with all decision nodes and loop back-edges ensures the cyclomatic complexity count — and therefore the number of independent paths selected for testing — matches the actual structural complexity of the code. Testing all independent paths exercises every branch and loop boundary condition (zero iterations, one iteration, many iterations), which statement- or line-based testing cannot guarantee, directly improving defect detection in both decision logic and loop handling.

Solution 4 — 100% Statement Coverage but Missed Logical Errors

Errors Identified

- The team used statement coverage as the sole adequacy criterion, which only confirms every line was executed at least once — it does not confirm every logical outcome of a condition was exercised.
- Conditional statements with compound logic (e.g., if (A && B)) were executed under only one combination of truth values, so logical errors in the condition itself went undetected.
- No criterion existed to confirm both the true and false branches of each decision were tested, let alone each condition within a compound decision.

Root Cause Analysis

Statement coverage is a weak criterion because a single test case can execute every statement in a function while only ever taking one path through its decisions (for example, always entering the 'true' branch of an if/else when both branches are on the same line count but only one is reached by the chosen inputs, or when a compound condition's component clauses are never independently varied). Mistaking 100% statement coverage for test adequacy is a classic gap between 'code was run' and 'logic was validated' — the team equated execution with verification.

Recommended Improvements

- Branch Coverage: ensure every decision point's true and false outcomes are each executed at least once.
- Condition Coverage: ensure each individual Boolean sub-condition (A, B in A && B) takes both true and false values across the test set.
- Modified Condition/Decision Coverage (MC/DC) for safety- or logic-critical modules: ensure each condition independently affects the decision outcome.

Justification

Branch and condition coverage force test cases to exercise both outcomes of every decision and sub-condition, directly targeting the logical-operator and comparison defects that statement coverage cannot see. Moving to MC/DC for critical modules further isolates the effect of each individual condition, catching subtle logic errors (e.g., wrong operator, inverted condition) that would otherwise reach production despite a 'fully covered' statement report.

Solution 5 — Incomplete Equivalence Classes for Age-Validation (0–100)

Errors Identified

- Only two classes were defined — "Negative numbers" and "Positive numbers" — which does not map to the actual specification of a valid range (0–100).
- Positive numbers above 100 are invalid but were grouped into the same class as valid positive numbers (1–100), mixing valid and invalid data.
- No boundary value analysis was performed at 0, 1, 100, and 101, the exact points where off-by-one defects occur.

Root Cause Analysis

The tester partitioned based on the mathematical sign of the number rather than the specification's actual valid range. This is a common mistake when the input type (integer) is confused with the business rule governing it (0–100 inclusive). Because "positive numbers" was treated as a single class, values like 150 or 9999 — which are positive but clearly invalid ages — were never isolated for testing, and the partitioning therefore does not reflect the real valid/invalid boundary structure of the field.

Corrected Partitions

- Invalid class 1: negative numbers (e.g., -1)

- Valid class: 0–100 (test a representative value, e.g., 45)
- Invalid class 2: numbers greater than 100 (e.g., 101, 500)
- Boundary values: -1 (just below valid), 0 (lower valid boundary), 100 (upper valid boundary), 101 (just above valid)

Justification

Classes built directly from the specified valid range (0–100) correctly separate valid data from both forms of invalid data (too low and too high), instead of relying on an unrelated property like sign. Adding the four boundary values targets exactly where comparison-operator defects (e.g., a field coded as age < 100 instead of age <= 100) would surface, giving far more robust input validation coverage than the original two-class design.

Solution 6 — Condition Coverage for a Compound Decision if(A && B)

Note: Question 6 in the assignment sheet is cut off in the source document (the condition statement and remaining instructions are incomplete, and no Bloom's Level is listed). The analysis below answers it based on the visible scenario — a developer claiming full Condition Coverage for the compound decision if(A && B) — and should be adjusted if the complete question text differs.

Errors Identified

- The developer likely tested only two combinations — A=True,B=True and A=False,B=False — which gives 100% statement and decision coverage but does not independently vary each condition, so it is not genuine condition coverage.
- Condition coverage requires each individual Boolean condition (A and B separately) to take both True and False values across the test set, which is not guaranteed by only testing the two 'matching' combinations.
- Combinations where A and B disagree (A=True,B=False and A=False,B=True) were not exercised, so errors in how the operator combines the two conditions could be masked.

Root Cause Analysis

The error stems from confusing decision coverage (every decision outcome, i.e., the overall if being True or False, is exercised) with condition coverage (every individual condition within the decision is exercised in both states). Testing A=T,B=T and A=F,B=F does achieve both overall outcomes of the decision, which can create the false impression of 'full' coverage, but it never isolates the effect of A or B individually — if the code mistakenly used || instead of &&, these two test cases alone might still pass.

Correction

A test set that achieves true condition coverage for if(A && B) must include all four combinations: (A=T,B=T), (A=T,B=F), (A=F,B=T), (A=F,B=F). This ensures A is tested True and False, and B is tested True and False, independently of each other.

Justification

Testing all four combinations exposes defects in the logical operator itself (e.g., && mistakenly coded as ||, or an inverted condition) because the two 'mixed' combinations (T,F and F,T) produce different expected outcomes under && versus other operators. This closes the gap left by decision-only coverage and gives much stronger assurance that the compound condition behaves exactly as specified. For full independence of effect (as required in safety-critical code), MC/DC would be the next step beyond condition coverage.

Solution 7 — Incomplete Decision Table for a Discount System

Errors Identified

- The decision table did not enumerate all combinations of the two Boolean conditions "Premium Member" and "Coupon Applied", so some rule combinations were silently left undefined or defaulted incorrectly.

- Because the conditions were not combined exhaustively, overlapping or conflicting discount rules were missed, producing incorrect price calculations for the untested combinations.
- No explicit "else/default" rule was captured for combinations the business analyst had not considered, leaving a gap between specification and implementation.

Root Cause Analysis

With two independent Boolean conditions, a complete decision table requires 2^2 = 4 rules. The team's table construction likely followed a narrative description of the discount policy ('premium members get X, coupon users get Y') rather than systematically enumerating every condition combination, so the combined case — premium member AND coupon applied — and/or the neither-applies case were never explicitly derived and tested. This is a common error when decision tables are written informally instead of generated from a truth-table style enumeration.

Corrected Decision Table

Conditions: Premium Member (Y/N), Coupon Applied (Y/N). Four rules:

- Rule 1: Premium=Y, Coupon=Y → apply combined premium + coupon discount (as per business rule, e.g., stacked or capped).
- Rule 2: Premium=Y, Coupon=N → apply premium discount only.
- Rule 3: Premium=N, Coupon=Y → apply coupon discount only.
- Rule 4: Premium=N, Coupon=N → no discount, standard price.

Justification

Enumerating all four rules guarantees every possible combination of inputs has a defined, tested expected outcome, eliminating the ambiguity that caused price-calculation errors. This exhaustiveness is decision table testing's core strength over ad-hoc rule writing: it mathematically guarantees no combination is overlooked, directly preventing the class of production defects described in the scenario (unexpected or incorrect pricing for combined-condition customers).

Solution 8 — Incomplete Loop Testing (n=1 and n=max only)

Errors Identified

- Only the minimum valid iteration count (n=1) and the maximum valid iteration count (n=max) were tested, skipping the critical zero-iteration case (n=0).
- The just-beyond-maximum case (n=max+1) was also skipped, so loop-termination defects (off-by-one errors in the loop-exit condition) were not caught.
- For any nested loops in the module, no combination of inner/outer boundary values was tested, leaving interactions between nested loop boundaries completely unexercised.

Root Cause Analysis

The tester applied a partial version of loop boundary testing, covering only the 'typical' valid boundaries (lowest and highest valid counts) while omitting the boundaries that sit just outside the valid range. Loop defects most commonly occur exactly at these exclusion boundaries — a loop that should skip execution entirely when n=0 but instead executes once, or a loop that should stop at n=max but continues to n=max+1 due to a <= vs < error. By not testing n=0 and n=max+1, the design left the most defect-prone boundary points completely unchecked.

Corrected Loop Testing Approach

- Simple loops: test n=0 (skip loop entirely), n=1 (minimum single pass), a typical interior value, n=max (maximum valid), and n=max+1 (just beyond maximum, expect rejection or graceful termination).
- Nested loops: starting from the innermost loop, hold outer loops at their minimum while testing the inner loop's full boundary set, then repeat for the outer loop's boundaries with the inner loop at a typical value, and finally test simultaneous minimum/maximum combinations of both loops together.

Justification

Including n=0 confirms the loop correctly handles the 'no iterations' case rather than assuming at least one pass, and including n=max+1 confirms the loop's exit condition is implemented correctly rather than allowing an extra, erroneous iteration. Extending this boundary set systematically to nested loops ensures that interactions between inner and outer loop limits — a common source of defects — are also verified, giving genuine path integrity rather than coverage of only the 'easy' middle cases.

Solution 9 — Sole Reliance on Error Guessing for a Safety-Critical Braking System

Errors Identified

- Error Guessing was used as the only testing technique for a safety-critical module, with no structured, specification-based black-box technique applied alongside it.
- Because error guessing depends entirely on the individual tester's experience and intuition, its coverage is unsystematic and unrepeatable — there is no guarantee that critical state transitions or boundary conditions of the braking system were exercised.
- No formal traceability exists between the braking system's safety requirements and the tests performed, which is unacceptable for a high-risk module typically subject to safety standards (e.g., ISO 26262 for automotive systems).

Root Cause Analysis

The root cause is treating an experience-based, exploratory technique as sufficient for a module where failure has safety consequences. Error guessing is valuable as a supplement — it catches defects that structured techniques might not anticipate — but it is inherently non-exhaustive and non-repeatable, since it relies on what the individual tester happens to think of. For a braking system, this is a critical strategic error: unexercised states (e.g., simultaneous brake and accelerator input, sensor failure during braking, ABS engagement thresholds) can remain completely untested if no tester happens to guess them.

Recommended Combination of Techniques

- Boundary Value Analysis (BVA): test brake-pressure thresholds, speed thresholds for ABS activation, and other numeric limits at and around their exact boundaries.
- State Transition Testing: model the braking system's states (idle, braking, ABS-engaged, brake-failure/fallback) and test every valid transition, every invalid transition (which should be rejected or trigger a safe fallback), and transitions triggered by boundary-level inputs.
- Error Guessing retained as a supplementary technique, applied after BVA and State Transition Testing to catch residual edge cases the structured techniques did not anticipate.

Justification

BVA guarantees the exact numeric thresholds governing braking behavior are deliberately and repeatably tested, while State Transition Testing guarantees every reachable (and every explicitly disallowed) system state and transition is verified against the specification — both essential for demonstrating safety assurance with full traceability. Combining these structured techniques with error guessing as a supplement, rather than a sole strategy, provides systematic, auditable coverage of the safety-critical requirements while still benefiting from the tester's intuition for unanticipated edge cases.

Solution 10 — Missing Else-Branches in a Control Flow Graph (0% Branch Coverage)

Errors Identified

- The CFG was constructed without nodes/edges for the "else" branches of the nested if statements, so the graph only represents the "true" path through each decision.

- Because the else branches are absent from the graph, test cases derived from it can reach 100% statement coverage (every statement in the true-branches is executed) while leaving every error/else path completely untested — hence 0% branch coverage on those paths.
- The independent paths calculated from this incomplete graph understate the true cyclomatic complexity, so the actual number of distinct execution paths through the nested decisions is undercounted.

Root Cause Analysis

This is a graph-construction error where only the 'positive' outcome of each if statement was modeled, likely because the else branches had no explicit statement in the source code (an implicit/empty else) and were therefore overlooked when drawing the CFG. However, in control flow terms, every if condition — whether or not it has an explicit else body — produces two outgoing edges (true and false) from its decision node; omitting the false-edge when the else is implicit is a conceptual gap, not a correct simplification. For nested ifs, this error compounds: each missing else edge also removes an entire sub-tree of downstream nodes and paths from the graph.

Corrected CFG and Independent Paths

For each if decision node (including nested ones), two outgoing edges must be drawn: one for the true branch (leading to its existing statements) and one for the false/else branch (leading either to an explicit else block or directly to the merge point following the if, if the else is implicit). Recomputing cyclomatic complexity V(G) = E – N + 2 on this corrected graph for, say, two nested ifs (yielding 4 decision outcomes) gives V(G) = 4, identifying 4 independent paths: both-true, true-then-false, false-then-true, and both-false — each corresponding to a distinct combination of branch outcomes that must have its own test case.

Justification

Explicitly modeling every false/else edge — even implicit ones — ensures the CFG reflects the code's true branching structure, so the independent paths derived from it include the error/else paths previously invisible to the testing team. Testing all four independent paths guarantees both branch outcomes of every nested decision are exercised, closing the exact gap that allowed 100% statement coverage to coexist with 0% branch coverage on error paths, and substantially improving defect detection in exception/fallback logic.